Conversão e Resolução de Expressões Numéricas usando Pilhas e Filas

Luiz Miguel Silvino

Curso de Ciência da Computação – UNIVALI
Itajaí – SC – Brasil

luizmiguelsilvino10@gmail. Com

Resumo. Este artigo descreve o desenvolvimento de uma aplicação em C++ capaz de analisar, converter para a Notação Polonesa Invertida (NPI) e solucionar expressões numéricas. Estruturas de dados lineares como Pilhas e Filas foram implementadas do zero, aplicando o algoritmo Shunting Yard para realizar a validação sintática e o cálculo sequencial. Os testes demonstram a eficácia da aplicação ao detectar erros de sintaxe e resolver expressões válidas de forma robusta.

1. Introdução

A avaliação de expressões matemáticas é um problema clássico na ciência da computação, essencial para o funcionamento de compiladores, calculadoras e sistemas de análise simbólica. Expressões numéricas, em sua notação infixa padrão (onde o operador fica entre os operandos), são facilmente compreendidas por humanos, mas apresentam ambiguidade computacional devido à precedência de operadores e ao uso de delimitadores como parênteses, colchetes e chaves.

Para contornar essa complexidade, utiliza-se a conversão para a Notação Polonesa Invertida (NPI), também conhecida como notação pós-fixa. Na NPI, os operadores sucedem seus operandos, eliminando a necessidade de parênteses e regras complexas de precedência durante a fase de avaliação. A estrutura ideal para gerenciar o escopo dos operadores e garantir a ordem correta das operações é a Pilha (estrutura LIFO - *Last In, First Out*), atuando em conjunto com a Fila (estrutura FIFO - *First In, First Out*) para organizar a saída.

Este trabalho tem como objetivo descrever a implementação de uma aplicação em linguagem C++ que processa expressões algébricas compostas por operadores aritméticos, números reais e variáveis. O programa realiza a verificação de integridade sintática e o cálculo numérico em múltiplas etapas, empregando Tipos Abstratos de Dados (TADs) construídos sem o uso de bibliotecas padrão.

2. Implementação e Algoritmos

O desenvolvimento foi segregado em três etapas fundamentais: o mapeamento de tipos de dados estruturados, o processamento e tradução da expressão via algoritmo *Shunting Yard* (idealizado por Dijkstra) e, finalmente, a substituição de variáveis e avaliação do resultado final.

2.1. Estruturas de Dados

Para garantir o pleno controle de alocação de memória e modularidade, implementou-se templates em C++ para as estruturas de `Pilha` e `Fila`. O projeto foi desenhado para aceitar a permuta entre alocação estática e dinâmica sem impacto nas funções principais. A estrutura base de trânsito de dados é o `Token`, um registro que armazena a natureza do fragmento lido (número, variável de uma letra ou operador aritmético), permitindo que a Fila da NPI armazene tipos heterogêneos de forma segura.

2.2. Análise Sintática e Conversão para NPI

A string de entrada é submetida a uma varredura sequencial. Quando dígitos contíguos são detectados, eles são aglutinados para compor valores numéricos de múltiplos dígitos e enviados diretamente à `filaNPI`. Delimitadores de abertura são empilhados na `pilhaOperadores`. Quando um fechamento é encontrado, a pilha é descarregada na fila até encontrar a abertura correspondente, momento no qual a função `combinaDelimitador` verifica o casamento exato (ex: '[' casando estritamente com ']').

Ao identificar operadores matemáticos (+, -, *, /, ^), o algoritmo compara a precedência do operador atual com o topo da pilha. Operadores de maior precedência, ou da mesma precedência com associatividade à esquerda, são removidos da pilha e enviados à fila antes de empilhar o operador atual. Variáveis de uma única letra são validadas e armazenadas em uma `TabelaVariavel` temporária, garantindo que o programa solicite o valor numérico dessa variável apenas uma vez ao final do processamento.

2.3. Resolução da Expressão

Uma vez montada a notação pós-fixa, a execução avança descarregando a fila. Se o token for um operando (número ou valor da variável fornecido pelo usuário), ele é empurrado para uma `pilhaValores` secundária. Se for um operador, a aplicação extrai (*pop*) os dois últimos valores da pilha, realiza a operação aritmética binária pertinente, protegendo contra divisão por zero, e empilha o resultado parcial. O processo finda com o resultado final isolado no topo da pilha.

3. Resultados e Análise Comparativa

Para atestar a precisão lógica do algoritmo, conduziu-se um teste baseado no caso de referência proposto na especificação do trabalho: a expressão infixa 3 + ((5 * a) - (b / (3 + c))). Durante a execução, o *parser* extraiu a variável "a" e assinalou o valor empírico 5, "b" recebeu 8 e "c" recebeu 1 . A Tabela 1 detalha os passos finais de avaliação com a Pilha.

Tabela 1. Rastreio de avaliação da Fila NPI e estado da Pilha

Token Atual	Ação Executada	Estado da Pilha (Base → Topo)
3 5 5	Empilha operandos	3, 5, 5
*	Pop(5, 5) → Multiplica (25)	3, 25
8 3 1	Empilha operandos	3, 25, 8, 3, 1
+	Pop(1 , 3) → Soma (4)	3, 25, 8, 4
/	Pop(4, 8) → Divide (2)	3, 25, 2
-	Pop(2, 25) → Subtrai (23)	3, 23
+	Pop(23, 3) → Soma (26)	26

4. Conclusão

A aplicação cumpriu com êxito os requisitos solicitados. A restrição do uso das bibliotecas de estruturas padrão da linguagem C++ permitiu o exercício da criação e manutenção de ponteiros e referências em estruturas lineares profundas. O motor de conversão identificou com precisão operadores mal posicionados e desbalanceamento de delimitadores (erros sintáticos). A utilização da Tabela de Variáveis reduziu o esforço do usuário ao evitar *prompts* duplicados, alcançando o cálculo de expressões complexas de maneira elegante, sequencial e matematicamente precisa.

Referências

GEEKSFORGEEKS. "Expression Evaluation using Stacks". https://www.geeksforgeeks.org/expression-evaluation/.

CPLUSPLUS. "Data Structures in C++". https://cplusplus.com/doc/tutorial/structures/.